Toy network demo

Table of contents

1. The network dataset 1
2. Build the tbnpy model 6
3. The RSR results 9
4. Sample the BN 9
5. The app 10
6. Hazard: the mock now, and what is expected 10
7. Files and expanding the test cases 13

This is an illustration using the toy network `toy-ca`. The only user input required is `network.json`, which is explained in Section 1. The rest of the demo is automated.

The system behind it is drawn in `schema/diagram.html`. Open it in a browser, or run `python -m src diagram --open`. It has three tabs:

- **Model:** every function, with the tables it reads.
- **Functions:** how each function's outputs feed the next one's inputs, from hazard to fragility, usability, edge and system state.
- **Plates:** the tables of `network.json` and their keys.

Clicking a card shows its signature and parameter sets.

1. The network dataset

The network is one file, `examples/toy-ca/network.json`, with one table per entity type.

```
data = json.loads(Path("examples/toy-ca/network.json").read_text(encoding="utf-8"))
{table: len(rows) for table, rows in data["tables"].items()}
```

```
{'Node': 8, 'Edge': 10, 'Bridge': 10, 'Event': 1, 'SystemEvent': 1}
```

One row of each table:

```
for table in ("Node", "Edge", "Bridge", "Event", "SystemEvent"):
    pprint({table: data["tables"][table][0]}, width=85, sort_dicts=False)
```

```
{'Node': {'id': 'n1',
          'lat': 36.2754377087278,
          'lon': -120.2697236285474,
          'key_destination': True}}
{'Edge': {'id': 'e1',
          'source': 'n1',
          'target': 'n2',
          'directed': False,
          'length_km': 106.011906,
          'capacity': 1,
```

```
          'avg_speed_kph': 60}}
{'Bridge': {'id': 'b1',
            'lat': 36.66025089106477,
            'lon': -120.62045329929501,
            'edge': 'e1',
            'hazus_class': 'HWB11'}}
{'Event': {'id': 'demo-san-andreas',
           'mw': 7.8,
           'lat': 36.5,
           'lon': -121.04,
           'depth_km': 10.0,
           'description': 'Hypothetical M7.8 on the San Andreas trace, at the '
                          "network's centre latitude"}}
{'SystemEvent': {'id': 'reach-key-destination',
                 'analysis': 'system.travel-time-to-nearest',
                 'target_add_h': 0.5,
                 'description': 'Each origin reaches its nearest key destination '
                                'within 0.5 h of its intact travel time.'}}
```

The tables:

```
import pandas as pd

nodes = pd.DataFrame(data["tables"]["Node"]).set_index("id")
edges = pd.DataFrame(data["tables"]["Edge"]).set_index("id")
bridges = pd.DataFrame(data["tables"]["Bridge"]).set_index("id")
nodes
```

id	lat	lon	key_destination
n1	36.275438	-120.269724	True
n2	37.045064	-120.971183	True
n3	36.830997	-121.387529	False
n4	36.503901	-121.127879	False
n5	36.177421	-120.827670	False
n6	36.112167	-121.022557	False
n7	36.409971	-121.328475	False
n8	36.666199	-121.645813	False

```
edges
```

id	source	target	directed	length_km	capacity	avg_speed_kph
e1	n1	n2	False	106.011906	1	60
e2	n1	n5	False	51.220587	1	60
e3	n2	n3	False	43.998194	1	60
e4	n3	n4	False	43.118452	1	60
e5	n3	n8	False	29.417095	1	60
e6	n4	n5	False	45.176621	1	60
e7	n4	n7	False	20.759042	1	60
e8	n5	n6	False	18.944186	1	60
e9	n6	n7	False	42.998648	1	60
e10	n7	n8	False	40.193808	1	60

```
bridges
```

id	lat	lon	edge	hazus_class
b1	36.660251	-120.620453	e1	HWB11
b2	36.226429	-120.548697	e2	HWB5
b3	36.938031	-121.179356	e3	HWB11
b4	36.667449	-121.257704	e4	HWB5
b5	36.748598	-121.516671	e5	HWB11
b6	36.340661	-120.977775	e6	HWB5
b7	36.456936	-121.228177	e7	HWB11
b8	36.144794	-120.925114	e8	HWB11
b9	36.261069	-121.175516	e9	HWB11
b10	36.538085	-121.487144	e10	HWB11

The schemas the file is checked against: its shape, from its `$schema`, and each table's columns, from `schema/entities.json`.

```
from jsonschema import Draft202012Validator

shape = json.loads(Path("schema/meta/network.schema.json").read_text(encoding="utf-8"))
print("$schema:", data["$schema"])
print("shape errors:", list(Draft202012Validator(shape).iter_errors(data)))
```

```
$schema: ../../schema/meta/network.schema.json
shape errors: []
```

```
entities = {e["id"]: e for e in
 ↪  json.loads(Path("schema/entities.json").read_text(encoding="utf-8"))["entities"]}

columns = []
for table, records in data["tables"].items():
    attrs = entities[table]["attributes"]
    in_file = set().union(*(r.keys() for r in records)) - {"id"}
    for name, spec in attrs.items():
        columns.append({"table": table, "column": name,
                        "schema type": f"-> {spec['ref']}" if "ref" in spec else
                         ↪  spec["type"],
                        "required": spec.get("required", True), "in file": name in in_file})
    for name in sorted(in_file - attrs.keys()):
        columns.append({"table": table, "column": name, "schema type": "NOT IN SCHEMA",
 ↪  "required": None, "in file": True})
columns = pd.DataFrame(columns)
columns
```

	table	column	schema type	required	in file
0	Node	lat	number	True	True
1	Node	lon	number	True	True
2	Node	population	number	False	False
3	Node	name	string	False	False
4	Node	key_destination	boolean	True	True
5	Edge	source	-> Node	True	True
6	Edge	target	-> Node	True	True
7	Edge	directed	boolean	True	True
8	Edge	length_km	number	True	True

	table	column	schema type	required	in file
9	Edge	capacity	integer	True	True
10	Edge	avg_speed_kph	number	True	True
11	Bridge	lat	number	True	True
12	Bridge	lon	number	True	True
13	Bridge	edge	-> Edge	True	True
14	Bridge	hazus_class	string	True	True
15	Bridge	vs30	number	False	False
16	Bridge	vs30_source	string	False	False
17	Event	mw	number	False	True
18	Event	lat	number	False	True
19	Event	lon	number	False	True
20	Event	depth_km	number	False	True
21	Event	description	string	False	True
22	SystemEvent	analysis	string	False	True
23	SystemEvent	target_add_h	number	False	True
24	SystemEvent	description	string	False	True

They match: every required column is in the file, and the file has no column the schema lacks.

```
missing = columns[columns.required.eq(True) & ~columns["in file"]]
extra = columns[columns["schema type"] == "NOT IN SCHEMA"]
print("required but missing:", missing[["table", "column"]].values.tolist())
print("in file, not in schema:", extra[["table", "column"]].values.tolist())
```

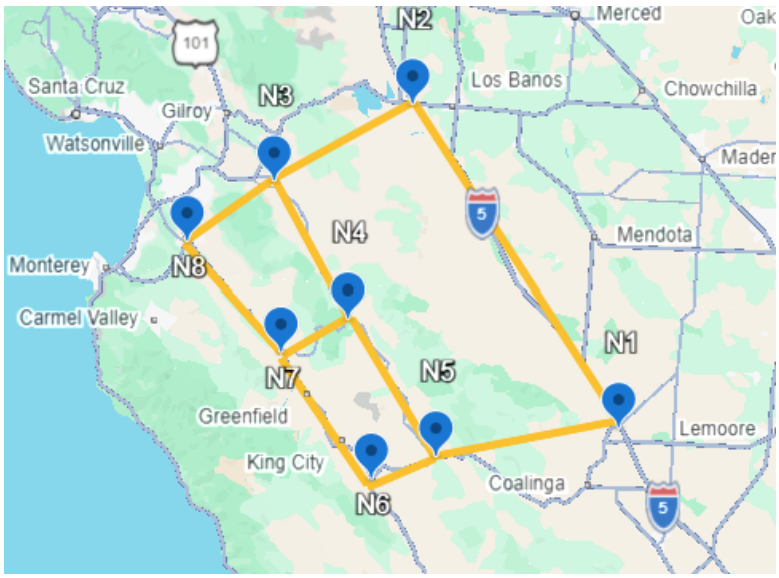
```
required but missing: []
in file, not in schema: []
```

Every assumed value is named in the file:

```
for key, text in data["assumptions"].items():
    print(fill(f"- {key}: {text}", width=90, subsequent_indent="  "))
```

```
- direction: All connections are treated as undirected.
- length_km: Great-circle distance between endpoint coordinates using a spherical Earth
  radius of 6371.0088 km; not road distance.
- capacity: Each edge has placeholder capacity 1 in arbitrary units; no measured
  capacities were supplied.
- bridge_representation: One bridge per edge; these are synthetic bridges for the toy
  example.
- bridge_location: Each bridge is placed at the arithmetic mean of its edge endpoints
  latitude and longitude; not a surveyed location.
- bridge_class: Seven bridges HWB11 (seismically designed continuous concrete); b2, b4 and
  b6, the three nearest the San Andreas trace, HWB5 (older, non-seismic) for the demo. Toy
  assumptions, not an inventory classification.
- avg_speed_kph: 60 km/h on every edge, assumed (the value the rsr and ndtools demos use);
  no measured speeds are in the dataset.
- key_destination: Key destinations n1 and n2, as set by the user.
- event: Hypothetical M7.8 as a point source on the San Andreas trace at 36.50 N, 121.04
  W, 10 km deep, for the demo only. This is the creeping section, not the 1857 rupture; a
  point source understates shaking along a ~300 km rupture.
- system_event: Survival is reaching the nearest key destination within 0.5 h of the
  intact network's travel time: the default threshold, assumed.
```

The map:




```
import matplotlib.pyplot as plt
import numpy as np

fig, ax = plt.subplots(figsize=(6, 5))
for _, e in edges.iterrows():
    a, b = nodes.loc[e.source], nodes.loc[e.target]
    ax.plot([a.lon, b.lon], [a.lat, b.lat], color="0.7", zorder=1)
ax.scatter(nodes.lon, nodes.lat, s=40, color="k", zorder=2, label="node")
for cls, group in bridges.groupby("hazus_class"):
    ax.scatter(group.lon, group.lat, s=35, marker="s", zorder=3, label=f"bridge {cls}")
key = nodes[nodes.key_destination]
ax.scatter(key.lon, key.lat, s=160, facecolors="none", edgecolors="tab:green", linewidths=2,
 ↪  zorder=4,
           label="key destination")
event = data["tables"]["Event"][0]
ax.scatter(event["lon"], event["lat"], marker="*", s=300, color="orange", zorder=5,
 ↪  label=f"scenario M{event['mw']}")
for nid, n in nodes.iterrows():
    ax.annotate(nid, (n.lon, n.lat), xytext=(4, 4), textcoords="offset points")
ax.set_aspect(1 / np.cos(np.radians(nodes.lat.mean())))
ax.set_xlabel("longitude")
ax.set_ylabel("latitude")
ax.legend(loc="lower left", fontsize=8)
plt.show()
```

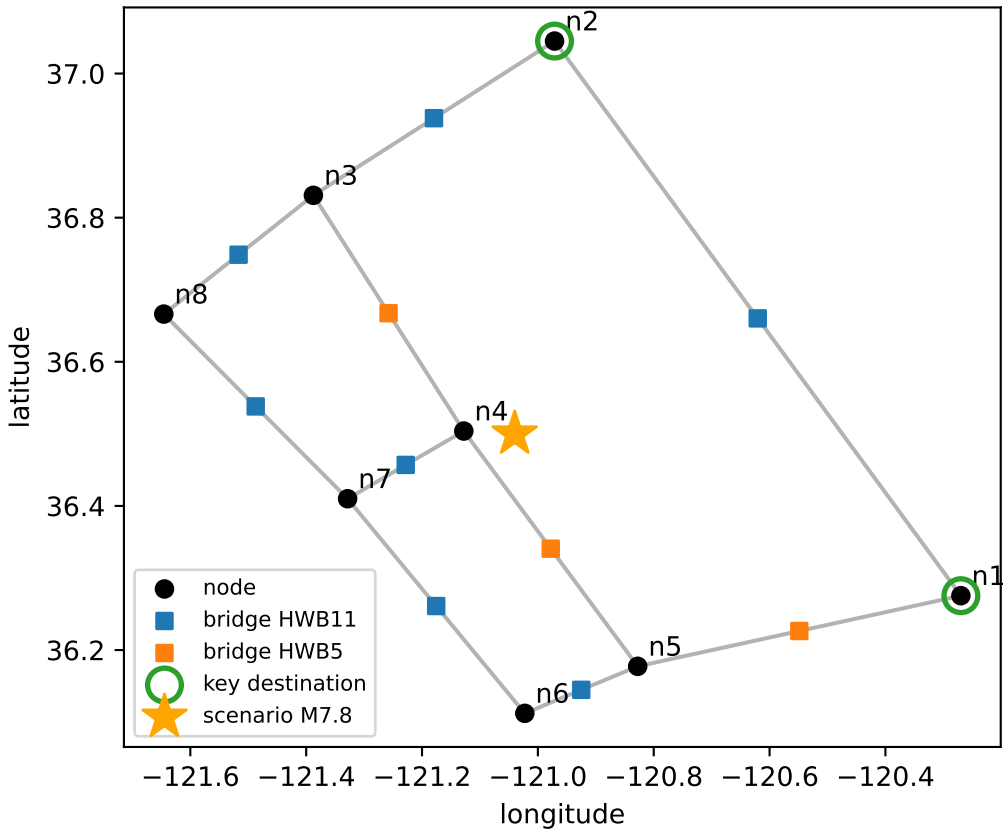


Figure 1: Nodes, edges, bridges by class, key destinations and the demo scenario.

Loading the network checks every row against `schema/entities.json`:

```
from src.builder.load import check_tables, load_library, load_network

lib = load_library()                        # schema/ and instances/
net = load_network("toy-ca", lib)
{entity: len(rows) for entity, rows in net.tables.items()}
```

```
{'Node': 8, 'Edge': 10, 'Bridge': 10, 'Event': 1, 'SystemEvent': 1}
```

A broken row is refused, with the reason:

```
broken = json.loads(json.dumps(net.tables))
broken["Bridge"][0]["edge"] = "e99"
broken["Edge"][0].pop("avg_speed_kph")
_, problems = check_tables(broken, lib, "toy-ca (edited)")
problems
```

```
["toy-ca (edited): Edge 'e1': missing required attribute 'avg_speed_kph'",
 "toy-ca (edited): Bridge 'b1': edge = 'e99' is not a Edge row"]
```

2. Build the tbnpy model

The builder resolves the assembly and saves the model and RSR rules under this example.

```
from src.builder.bn_builder import (
    generate_tbn, define_tbn_variables, define_tbn_probabilities,
)

model_dir = generate_tbn(Path("examples/toy-ca/demo.assembly.json"))
```

Load the saved data into tbnpy's two dictionaries:

```
model_data = json.loads((model_dir / "model.json").read_text(encoding="utf-8"))
varis = define_tbn_variables(model_data)
probs = define_tbn_probabilities(model_data, varis, directory=model_dir)
print(f"{len(varis)} variables; {len(probs)} probability objects")
```

```
46 variables; 37 probability objects
```

`varis` holds scalar variables, grouped below. The scenario (the Event row) is fixed data read by the hazard object, so it is not a variable.

quantity	count	type
SA(1.0)	10	continuous
DamageState.HazusBridge	10	discrete
Usable	10	discrete
EdgeOpen	10	discrete
SystemState	6	discrete

Total: **10 continuous** and **36 discrete** variables.

`probs` holds the probability objects, each with `childs`, `parents`, `sample()` and `log_prob()`. Each variable has its own object, keyed by the variable, except the ten intensities: the hazard field draws them together.

```
by_function = {}
for k, p in probs.items():
    by_function.setdefault(p.spec["function"], []).append((k, p))

def span(values):
    return str(min(values)) if min(values) == max(values) else
     ↪  f"{min(values)}-{max(values)}"

show_table(pd.DataFrame([
    {"function": f, "objects": len(ps),
     "children each": span([len(p.childs) for _, p in ps]),
     "parents each": span([len(p.parents) for _, p in ps]),
     "example key": ps[0][0]}
    for f, ps in by_function.items()
]))
```

function	objects	children each	parents each	example key
hazard.gmm	1	10	0	hazard.gmm
fragility.bridge	10	1	1	DamageState.HazusBridge::b1
functionality.usable-from-damage	10	1	1	Usable::b1
edge.open	10	1	1	EdgeOpen::e1
system.travel-time-to-nearest	6	1	10	SystemState::n3

The graph groups variables by quantity. Arrows show the conditional direction; each system state takes every edge state as input.

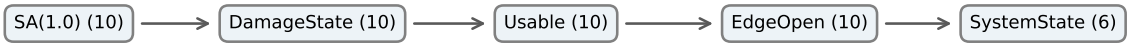


Figure 2: Variable families and their counts. The intensities share one joint object; every other variable has its own.

One instance of each type

In tbnpy, a probability object's `parents` are its inputs and `childs` are its outputs. The examples below follow bridge b4, edge e4 and origin n4.

Intensity — **`SA(1.0)::b4`.** No parents: the event's Mw, epicentre and depth are fixed data from `network.json`. Outputs: all ten bridge intensities jointly; b4's output feeds `DamageState.HazusBridge::b4`. The Campbell function calculates log-median μ and dispersion σ, then draws $I = \exp(\mu + \sigma Z)$ with $Z \sim N(0, 1)$. This demo supplies only SA(1.0), with independent residuals between bridges.

For b4: median = 0.282 g; log-dispersion = 0.620; sampled intensity = 0.200 g.

Damage — **`DamageState.HazusBridge::b4`.** Parent: `SA(1.0)::b4`. Child: `DamageState.HazusBridge::b4`, which feeds `Usable::b4`. Lognormal exceedances are differenced into five state probabilities, then one uniform draw per bridge picks a state index. Indices 0–4 denote none through complete.

damage state	P(state given sampled intensity)
none	0.645
slight	0.179
moderate	0.087
extensive	0.070
complete	0.018

Sampled damage at b4: index 0, state **none** (no damage).

Usability — **`Usable::b4`.** Parent: `DamageState.HazusBridge::b4`. Child: `Usable::b4`, which feeds `EdgeOpen::e4`. The closure policy deterministically maps none/slight/moderate to 1, and extensive/complete to 0.

Edge state — **`EdgeOpen::e4`.** Parents: the usability of every bridge on e4, here `Usable::b4`. Child: `EdgeOpen::e4`. An edge is open only if every bridge on it is usable. Every edge state feeds each origin's system state, n3–n8.

For example, this calls the edge object's sampling method directly. The tensor columns follow `parents`; returned columns follow `childs`.

```
edge = probs["EdgeOpen::e4"]
Cs_pars = example_parents(edge)
Cs_child, logp = edge.sample(Cs_pars=Cs_pars)
print("parents:", [v.name for v in edge.parents], "; childs:", [v.name for v in
 ↪  edge.childs])
print("b4 usability:", int(sampled_value("Usable::b4")))
print("e4 state:", int(Cs_child[0, 0]), "; probability:", float(np.exp(logp[0])))
```

```
parents: ['Usable::b4'] ; childs: ['EdgeOpen::e4']
b4 usability: 1
e4 state: 1 ; probability: 1.0
```

System state — `SystemState::n4`. Parents: all ten edge states. Child: `SystemState::n4`, with no downstream variables; each of the six origins has its own object. RSR first classifies the edge configuration; ndtools handles any unknowns. Survival means n4 reaches n1 or n2 within 0.5 hours of its intact travel time, with edge time = length/speed.

Sampled state of n4: **1**.

3. The RSR results

Each origin has survival and failure rules under `rsr/<origin>/target_add_h=0.5/`, the network's threshold. The counts below also check rule coverage across all 1,024 edge configurations.

origin	survival rules	failure rules	rounds	unclassified / 1024
n3	1	1	3	0
n4	2	4	7	0
n5	1	1	3	0
n6	1	2	4	0
n7	4	16	20	6
n8	1	2	4	0

For example, these rules determine whether n4 stays within 0.5 hours of its intact travel time. `1` means open; `0` means closed. Unlisted edges are unrestricted.

```python
for filename, outcome in (("refs_up_1.json", "survives"),
                          ("refs_low_0.json", "fails")):
    rules = json.loads((model_dir / "rsr/n4/target_add_h=0.5" / filename).read_text())
    for rule in rules:
        conditions = " and ".join(f"{edge}={value}" for edge, (_, value) in rule.items()
                                  if edge != "sys")
        print(f"{conditions} -> n4 {outcome}")
```

```
e3=1 and e4=1 -> n4 survives
e2=1 and e6=1 -> n4 survives
e3=0 and e6=0 -> n4 fails
e2=0 and e3=0 -> n4 fails
e2=0 and e4=0 -> n4 fails
e4=0 and e6=0 -> n4 fails
```

RSR classifies the BN's edge samples; ndtools resolves any unclassified configurations. The 50/50 edge probabilities used during rule search are not the BN's probabilities.

4. Sample the BN

tbnpy samples the system state and all its ancestors using PyTorch. These are Monte Carlo estimates under the demo scenario.

```python
from tbnpy import inference

torch.manual_seed(20261022)
result = inference.sample(probs, state_keys, n_sample=5000)
show_table(pd.DataFrame([
    {"origin": key.split("::")[1],
```

```
     "P(fails)": f"{(result[key].Cs[:, 0] == 0).double().mean().item():.4f}"}
    for key in state_keys
]))
```

origin	P(fails)
n3	0.0102
n4	0.1434
n5	0.0864
n6	0.1072
n7	0.0026
n8	0.0206

Sampling time

CPU timings for the same six system states and their ancestors, using batches of 50,000. Each time is the median of three runs after a warm-up; model building and RSR extraction are excluded. P(fails) for n7 and n8 is from the first run at each size.

PyTorch 2.14.1+cpu; 1 CPU threads; float64.

samples	median time (s)	samples/s	P(n7 fails)	P(n8 fails)
100,000	0.727	137,573	0.0028	0.0210
200,000	1.515	132,044	0.0028	0.0215
300,000	2.353	127,489	0.0029	0.0217
400,000	3.097	129,177	0.0029	0.0216
500,000	3.799	131,624	0.0028	0.0216
600,000	4.680	128,196	0.0029	0.0217
700,000	5.514	126,951	0.0029	0.0217
800,000	6.258	127,843	0.0029	0.0217
900,000	6.966	129,206	0.0029	0.0217
1,000,000	7.614	131,344	0.0029	0.0217

The individual timings and settings are saved in `tbnpy/demo/sampling_benchmark.json`.

5. The app

Run `python -m src serve --example toy-ca/demo --open`. The Map tab combines scenario controls, the network, results and Ask; BN and Schema show the model. Ask translates words into validated queries, reads them back, then gives the engine's results and provenance. Optional query review pauses before sampling.

The engine supports forced discrete node states, joint events and discrete observations. Comparisons, bridge importance and candidate-path rankings are computed in Python. Inspection priority uses value of information from supplied actions, losses, accuracy and cost. Missing inputs prompt a clarification. Evidence with too few matching samples is refused; continuous evidence and sequential inspection policies remain outside this pilot.

The Proposals tab reviews new parameter sets and saved scenario variants. Drafts include sources and validation results; a named user activates the reviewed version. New mathematical forms still need code and tests.

6. Hazard: the mock now, and what is expected

The hazard node generates **a joint sample of the intensity measure at every bridge.** There are two modes that differ in what supplies it.

Mode	Source	Status
Preparation	OpenQuake scenario: rupture + GMM	**to build**; a mock stands in
Emergency	ShakeCast-triggered USGS ShakeMap	built; needs connecting to live ShakeCast

The contract

A hazard function is one probability object over all bridges. The current mock looks like:

```
hazard = probs["hazard.gmm"]
Cs, logp = hazard.sample(n_sample=4)
print("childs:", [v.name for v in hazard.childs[:3]], "...", len(hazard.childs))
print("parents:", hazard.parents)
print("sample ->", tuple(Cs.shape), "IMs in g;", tuple(logp.shape), "log probabilities")
print("notes:", fill(hazard.notes[0], 90))
```

```
childs: ['SA(1.0)::b1', 'SA(1.0)::b2', 'SA(1.0)::b3'] ... 10
parents: []
sample -> (4, 10) IMs in g; (4,) log probabilities
notes: hazard.gmm draws each Bridge's residual independently: no spatial correlation, so joint
outcomes across the network are likely understated.
```

Item	Requirement
`childs`	`<IMT>::<bridge id>` for every bridge, e.g. `SA(1.0)::b4`
`sample(n_sample=n)`	`(n, bridges)` tensor of IMs (g, or cm/s for PGV), plus `(n,)` log probabilities
`log_prob(Cs)`	log density of given samples, consistent with `sample`
`imt`	the same IM as the fragility curves (`SA(1.0)` for Hazus); `check` refuses a mismatch
`notes`	provenance shown with every answer: model, map version, assumptions
fixed data	the `Event` row and each bridge's `lat`/`lon` (and `vs30` when the GMM needs it)

The mock (stands in for preparation mode)

The scenario is the `Event` row of `network.json`. The GMM is a parameter set chosen by location. The demo set reuses Campbell (2003)'s eastern North America coefficients over a box drawn around toy-ca. Its residuals are independent between bridges.

```
demo_gmm = lib.parameter_sets["demo.campbell-2003-in-california"]
print("Event:", {k: v for k, v in data["tables"]["Event"][0].items() if k != "description"})
print("set:", demo_gmm["label"])
print({k: demo_gmm["values"][k] for k in ("form", "imt", "sigma_epistemic",
 ↪  "magnitude_range")})
print("region:", demo_gmm["region"]["name"], "[lon, lat] box:",
 ↪  demo_gmm["region"]["polygon"][0], "to", demo_gmm["region"]["polygon"][2])
```

```
Event: {'id': 'demo-san-andreas', 'mw': 7.8, 'lat': 36.5, 'lon': -121.04, 'depth_km': 10.0}
set: DEMO ONLY: Campbell (2003) ENA coefficients applied in central California
{'form': 'campbell-2003', 'imt': 'SA(1.0)', 'sigma_epistemic': 0.3, 'magnitude_range': [4.0, 9.5]}
region: Central California (demo) [lon, lat] box: [-122.5, 35.5] to [-119.5, 37.5]
```

Preparation mode: OpenQuake (expected)

Where it goes. A new function, say `hazard.openquake-scenario`. Its class lives in `src/builder/kernels/hazard.py`, and it is the same kind of object as `Gmm`: per `Bridge`, `drawKind: Joint`, output `{imt}`, reading the `Event` table and each bridge's `lat`, `lon` and `vs30`.

What OpenQuake supplies. `openquake.hazardlib` gives each site's mean ln IM and the between-event (τ) and within-event (ϕ) standard deviations, for the rupture and the GMM.

What we sample. We sample the field ourselves; it is fast enough for 10^6 draws. η is one draw shared by every bridge. ε is correlated between bridges by the existing `hazard.spatial-correlation` model (Jayaram & Baker 2009).

$$\ln I_b = \mu_b + \tau_b\,\eta + \phi_b\,\varepsilon_b$$

Instances. A parameter set under `instances/hazard/` names the GMM and its IM, e.g. `{"gsim": "...", "imt": "SA(1.0)"}`, with a `region` so `parameter_set_by_location` can choose it.

Data.

- `Event` gains what an OpenQuake rupture needs, e.g. strike, dip, rake, or a path to a `rupture.xml`.
- `Bridge.vs30` takes the user's value where given. Missing values are filled with `python -m src vs30 <network>` from the USGS mosaic. That command is untested: it needs GDAL and the grid downloaded by hand.

Several IMs (e.g. PGA and SA(1.0)) stay in the same object, so their correlation is kept.

Swapping the mock for it is one binding in the assembly; nothing else changes:

```
{"function": "hazard.gmm", "parameter_set_by_location":
 ↪  ["demo.campbell-2003-in-california"]}
{"function": "hazard.openquake-scenario", "parameter_set_by_location": ["<oq set for
 ↪  California>"]}
```

The rules extracted in section 3 still apply after the swap, because they depend on the network and the system event, not the hazard. The same holds for a changed scenario:

```
from src.builder.bn_builder import build_tbn
from src.inference.engine import compile_query

_, other = build_tbn(compile_query({"assembly": "toy-ca/demo",
                                    "rows": {"demo-san-andreas": {"mw": 7.0}}}))
print("rules reused at M7.0:", all(p.refs[0] is not None
                                   for p in other.values() if p.spec["role"] == "system"))
```

```
rules reused at M7.0: True
```

Emergency mode: ShakeCast to ShakeMap (in place)

Two functions already do this, as in `hebei-transport/shakemap`:

- `hazard.spatial-correlation` draws the residual field ε jointly over all bridges.
- `hazard.shakemap` reads the median from `grid.xml` and σ from `uncertainty.xml` at each bridge, and sets $\ln I = \ln \text{median} + \sigma\varepsilon$.

```
hebei = json.loads(Path("examples/hebei-transport/shakemap.assembly.json"
 ↪  ).read_text(encoding="utf-8"))
for b in hebei["bindings"][:2]:
    pprint({k: v for k, v in b.items() if k != "note"}, width=90)
```

```
{'function': 'hazard.spatial-correlation', 'parameter_set': 'jayaram-baker-2009.sa1p0'}
{'function': 'hazard.shakemap',
 'parameters': {'event_id': 'iscgem849480',
                'imt': 'SA(1.0)',
                'source': None,
                'version': None}}
```

Where maps come from.

- **ShakeCast first:** its data directory (`SHAKECAST_USER_DIRECTORY`, default `~/.shakecast`), laid out as `<event>/<event>-<version>/grid.xml`.
- **USGS catalogue otherwise.**
- **Choosing:** `BN_SHAKEMAP_SOURCE` = `auto`, `shakecast` or `comcat`.
- **Caching:** grids are cached in `.cache/shakemaps/`.
- **Outside the map:** a bridge outside the map is refused, not set to zero.

What is expected.

- **ShakeCast's trigger:** it should hand the new `event_id` to the assembly, e.g. `--set` or the query's `parameters`: `{"hazard.shakemap": {"event_id": "..."}}`.
- **The live server:** the directory layout and the grid fields (`PSA10`/`STDPSA10` for SA(1.0), and so on) should be confirmed against a live ShakeCast server.
- **A toy-ca assembly for this mode:** `toy-ca/shakemap` would use a California scenario ShakeMap, so both modes can be compared on the same network.

Checks a new hazard must pass

- `python -m src check`: its schema file and instances validate, and its IM wires to the curves.
- `sample` and `log_prob` agree, as `test_samples_and_log_probabilities_agree` checks for the current objects.
- A closed-form check on one bridge, like `test_event_probability_matches_the_closed_form` for the GMM.

Other potential useful additions

- Showing the rupture locations or other hazard information in the map, overlaid with the network.

7. Files and expanding the test cases

The answer files below live in `examples/toy-ca/`; code paths start at the repository root.

Files	Role
`answer-sheet.md`, `answer-cases/*.json`	Agreed meanings and executable expectations: context, phrasings, tools and forbidden interpretations.
`src/llm/chat.py`, `cases.py`	Translation, tools, few-shot selection and held-out evaluation.
`src/inference/engine.py`, `query.schema.json`	Sampling, interventions, joint events and evidence.
`comparisons.py`, `decision.py`, `records.py` in `src/inference/`	Comparisons and paths, VOI, and provenance. New numerical capabilities need tests.
`schema/`, `instances/`, network and assembly JSON	Allowed structures, sourced values and their application. New forms need kernels and tests.
`src/proposals.py`, `proposals/`	Draft validation and reviewed activation.
`src/app/server.py`, `quakenet.html`	API and interface for new tools and results.

Copy a case JSON, give it a new ID, and vary networks, origins, destinations, thresholds, evidence, interventions and wording. Include missing-input questions; distinguish inspection from repair and observation from intervention. Keep held-out cases out of prompts.

`python -m src evaluate` checks contracts; `--live` logs held-out LLM translations without computing. Review semantic alternatives; contract checks alone are not LLM accuracy.